Enterprise Angular Authentication

Dual-Token Security Architecture & Workflow

Gym Tracking Application Documentation

1 The Cast of Characters

To understand the architecture, we map each file to a real-world concept:

File / Module	Architectural Role
`core/models/*.model.ts`	**The Data Blueprints**: Defines strict TypeScript shapes (e.g., `ApiResponse<T>`, `User`) ensuring frontend and backend contracts match exactly.
`app.config.ts`	**The Main Breaker Box**: Initializes the Router and attaches the `authInterceptor` to the global `HttpClient`.
`app.routes.ts`	**The Floor Plan**: Maps URLs to lazy-loaded components and assigns specific Guards to protect each route.
`core/guards/*.guard.ts`	**The Door Bouncers**: Functions (`authGuard`, `adminGuard`) that query the `AuthService` before allowing a component to load.
`core/services/auth.service.ts`	**The Front Desk**: A singleton that manages communication with the backend, stores tokens in `localStorage`, and broadcasts identity via the `currentUser` Signal.
`core/interceptors/...`	**The Automated Toll Booth**: Intercepts all outgoing HTTP traffic, staples the Bearer token, and handles 401/403 queueing.

2 The Dual-Token Strategy

- **Access Token (15-Minute JWT)**: A short-lived credential sent via the `Authorization: Bearer` header on every standard API request. Its short lifespan mitigates the risk of interception.
- **Refresh Token (7-Day UUID)**: A long-lived credential stored securely and used exclusively to request new Access Tokens via `POST /api/v1/users/refresh` when a 401 Unauthorized response occurs.

3 Core Workflows

3.1 Cold Start & The Guard Block

When a guest visits `http://localhost:4200/`, the Router triggers `authGuard`. The guard checks `authService.currentUser()`. Because `localStorage` is empty, it returns `null`, blocking access and redirecting to `/login`.

3.2 The Silent Refresh (Concurrency & Race Condition Prevention)

If a user submits two concurrent requests (e.g., `/me` and `/programs`) after their 15-minute token expires, both fail with `401 Unauthorized`. The `authInterceptor` catches this:

1. **Request 1** locks the mutex (`isRefreshing = true`), nullifies the `BehaviorSubject` queue, and fires the `/refresh` request to Spring Boot.
2. **Request 2** sees the locked mutex, pauses, and subscribes to the `BehaviorSubject` waiting room.
3. Spring Boot returns new tokens. The interceptor posts the new JWT to the waiting room. Both requests automatically clone themselves with the new token and replay successfully.

4 Visual Architecture Map

The following diagram illustrates the unidirectional data flow and security checkpoints from the browser URL down to the PostgreSQL database.

